

CLOUD-BASED HEALTHCARE APPOINTMENT BOOKING SYSTEM

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Bethala Sumana, APSSDC
Mr. Juttuka Lovababu, APSSDC

Submitted by

Gavini Divya Madhuri

Gumma Ankitha

Saketi Bhanu Vighnesh

Nelakurti Maneesha

Velpuri Yasaswini

Acknowledgement

We would like to express our heartfelt gratitude to all those who have contributed to the successful completion of our summer internship project at Andhra Pradesh Skill Development Corporation (APSSDC). This opportunity has been an enriching and transformative experience for us, and we are truly thankful for the support, guidance, and encouragement we have received along the way.

First and foremost, we extend our sincere regards to Mrs. Bethala Sumana and Mr. Juttuka Lovababu, our supervisors and mentors, for providing us with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

We would like to thank the entire team at Andhra Pradesh Skill Development Corporation (APSSDC) for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback we received from our colleagues significantly contributed to the development and success of this project.

In conclusion, we are honoured to have been a part of this internship program, and we look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Gavini Divya Madhuri

Gumma Ankitha

Saketi Bhanu Vighnesh

Nelakurti Maneesha

Velpuri Yasaswini

Abstract

This project presents the design and implementation of a Cloud-Based Healthcare Appointment Booking System using Amazon Web Services (AWS) to automate and simplify the process of scheduling doctor appointments. The primary goal is to replace slow, manual, walk-in and phone-based booking with a fast, reliable, and scalable online platform that works for hospitals of any size.

The system allows patients and doctors to securely register and log in, after which patients can search for doctors by specialty and view real-time slot availability. Once a patient selects a convenient time, the system automatically checks for scheduling conflicts before confirming the booking, and an instant email or SMS notification is sent to confirm the appointment.

The solution is built on a serverless, cloud-native architecture using Amazon Cognito for secure authentication, Amazon API Gateway as the entry point for all requests, AWS Lambda for business logic, Amazon DynamoDB for storing appointment and patient data, Amazon S3 for document storage, and Amazon SES/SNS for automated notifications. This architecture eliminates the need for provisioned servers, scales automatically with demand, and reduces operational overhead.

By automating booking, eliminating double-booking errors, and giving patients real-time visibility into doctor availability, the system significantly improves the patient experience while reducing the administrative burden on hospital staff. The project demonstrates how cloud-native, serverless design can be applied to solve real-world healthcare operations challenges, with a clear path toward future enhancements such as mobile apps, telemedicine, and AI-based doctor recommendations.

Table of Contents

1. INTRODUCTION	5
2. METHODOLOGY.....	6
3. TECHNOLOGY STACK.....	8
4. SYSTEM DESIGN / ARCHITECTURE.....	10
5. IMPLEMENTATION	15
6. RESULTS.....	17
7. CONCLUSION.....	19

1. Introduction

In today's fast-paced healthcare environment, timely access to medical consultation is a critical factor in patient satisfaction and care outcomes. This project, titled Cloud-Based Healthcare Appointment Booking System, leverages the scalability and reliability of Amazon Web Services (AWS) to deliver a modern, automated platform for booking doctor appointments online — replacing traditional walk-in and phone-based scheduling with a fast, transparent, and always-available digital experience.

Hospitals and clinics today face several recurring operational challenges. Manual, walk-in, and phone-based appointment booking is slow and inconvenient for patients, often resulting in long waiting queues and overcrowded reception areas. Without a centralized system, patients have no easy way to check real-time doctor availability across departments, and without automated conflict checks, hospitals frequently struggle with double-booking errors that frustrate both patients and staff.

The Cloud-Based Healthcare Appointment Booking System addresses these challenges directly. Patients and doctors can securely create accounts and log in using Amazon Cognito. Patients can then search for doctors by specialty and instantly view free time slots. When a patient selects a slot, the system automatically verifies availability to ensure no two patients are assigned the same appointment, and an instant email or SMS confirmation is sent the moment the booking is finalized. All patient, doctor, and appointment data is stored securely on the cloud.

What sets this project apart is its use of a fully serverless AWS architecture, which removes the need for hospitals to provision or maintain physical servers. Each component of the system — authentication, request routing, business logic, storage, and notifications — scales independently and automatically based on demand, keeping the platform fast and reliable even as the number of patients and hospitals grows.

Key Features

- **Secure Login:** Only registered, verified patients and doctors can log in to the platform, protecting sensitive medical data.
- **Doctor Search:** Patients can easily find a doctor by specialty and instantly view their free time slots.
- **Instant Booking:** Patients can book an appointment in a single click, with no waiting and no scheduling clashes.
- **Auto Notifications:** Patients and doctors receive an email or SMS the moment a booking is confirmed.
- **Health Records:** Past appointments are saved and easy for patients and doctors to review later.

- **Runs on Cloud:** Built on AWS so the platform performs smoothly for many simultaneous users.

This report provides a comprehensive overview of the system's design, architecture, and implementation, highlighting the technical decisions and AWS integrations that make the platform secure, scalable, and easy to use for both patients and healthcare providers.

2. Methodology

The development of the Cloud-Based Healthcare Appointment Booking System followed a structured and iterative methodology to ensure the project met both functional and technical objectives. The methodology was divided into key phases: requirements gathering, system design, implementation, testing, deployment, and refinement, aligned with best practices for cloud-native application development.

2.1. Requirements Gathering

The project began with an analysis of the day-to-day scheduling challenges faced by hospitals and patients. Discussions with mentors helped define the core objectives: allow patients to search doctors and view real-time availability, automate conflict-free appointment booking, and deliver instant confirmation notifications — all through a secure, cloud-hosted platform.

2.2. Design

Based on the identified requirements, a cloud-native, event-driven architecture was designed using AWS managed services. The workflow was defined to include a frontend web application, Amazon Cognito for authentication, Amazon API Gateway as the request entry point, AWS Lambda for booking logic, DynamoDB for appointment storage, S3 for documents, and SES/SNS for notifications.

2.3. Implementation

The implementation phase translated the design into a functioning application. The frontend was connected to API Gateway endpoints secured by Cognito authentication. Lambda functions were developed to handle registration, doctor search, slot availability checks, and booking confirmation, with all appointment records written to DynamoDB and confirmation messages dispatched through SES/SNS.

2.4. Database Management

A NoSQL approach was adopted using Amazon DynamoDB for fast, flexible storage of patient, doctor, and appointment records. The schema was designed to support quick lookups of doctor availability and patient booking history, while S3 was used to store supporting documents such as medical reports and profile photos.

2.5. Testing

Multiple levels of testing were carried out to validate booking accuracy, conflict detection, and notification delivery. Unit tests validated individual Lambda functions, while integration testing confirmed that Cognito, API Gateway, Lambda, DynamoDB, S3, and SES/SNS worked together

correctly across the full booking flow. Functional testing simulated concurrent bookings to confirm the system reliably prevents double-booking.

2.6. Deployment

The system was deployed using AWS-managed services, with IAM roles configured to grant least-privilege access between components. CloudWatch was used to monitor Lambda executions, track errors, and ensure the platform operates reliably with minimal manual intervention.

2.7. Iterative Refinement

Throughout development, feedback loops were used to refine the booking flow, notification formatting, and error handling. The architecture was reviewed periodically to ensure it followed serverless best practices, keeping the system maintainable and ready for future enhancements such as telemedicine and mobile app support.

The methodology adopted for this project ensured a cloud-native, scalable, and patient-friendly solution, successfully demonstrating how serverless AWS architecture can modernize healthcare appointment scheduling.

2.8. Documentation and Team Collaboration

Throughout the internship, the team maintained detailed documentation covering system requirements, architecture diagrams, API contracts, and Lambda function specifications, ensuring that every design decision was traceable and easy to hand over. Regular check-ins with the mentors, Mrs. Bethala Sumana and Mr. Juttuka Lovababu, helped validate progress at each phase and kept the implementation aligned with real-world hospital workflows. Version control practices were followed for all source code and configuration files, allowing changes to be tracked, reviewed, and rolled back safely whenever an issue was discovered during testing. This disciplined approach to documentation and collaboration ensured that the final system was not only functionally correct but also maintainable and easy to extend in future iterations of the project.

3. Technology Stack

The Cloud-Based Healthcare Appointment Booking System was built using a modern, cloud-native technology stack supporting secure authentication, event-driven processing, and real-time data access. The following technologies and services were used throughout the implementation:

3.1. Cloud Platform

- **Amazon Web Services (AWS):** The entire platform is hosted on AWS, leveraging its managed services for authentication, compute, storage, database, and notifications.

3.2. AWS Services Used

AWS Service	What It Does
Amazon Cognito	Checks login and manages secure identity for patients and doctors — like a security guard for the app.
Amazon API Gateway	The door that connects the frontend web app to the backend booking logic.
AWS Lambda	Runs the core code that books the appointment — no server needed.
Amazon DynamoDB	The database that stores all patient and appointment details.
Amazon S3	Cloud storage that keeps documents and profile photos.
Amazon SES / SNS	Sends the confirmation email and SMS to the patient.
AWS IAM	Grants least-privilege roles and permissions governing all inter-service access.
Amazon CloudWatch	Monitors Lambda invocations, execution duration, and errors for full operational visibility.

3.3. Frontend & Programming

- **Frontend Web App:** A responsive web application through which patients and doctors interact with the platform — searching for doctors, viewing availability, and managing bookings.
- **Python (Lambda Runtime):** Used for writing the serverless backend logic in AWS Lambda functions.
- **Boto3 SDK:** Python SDK for AWS used within Lambda to interact with Cognito, DynamoDB, S3, and SES/SNS.

3.4. Security & Access

IAM Roles and Policies: Configured to allow only the necessary permissions for each service, following the principle of least privilege. Amazon Cognito enforces secure, verified access for both patients and doctors.

3.5. Monitoring & Logging

- **CloudWatch Logs:** Captures execution logs from Lambda for tracking and troubleshooting.
- **CloudWatch Metrics:** Provides real-time data on function invocations, errors, and duration.

4. System Design / Architecture

The Cloud-Based Healthcare Appointment Booking System is designed as a secure, event-driven, cloud-native solution that automates the entire appointment booking journey — from patient login to instant confirmation. The architecture follows a multi-tier design comprising the presentation layer, authentication layer, API layer, processing logic, data storage, and notification layer, all powered by managed AWS services.

This section outlines the core components of the architecture and how they interact.

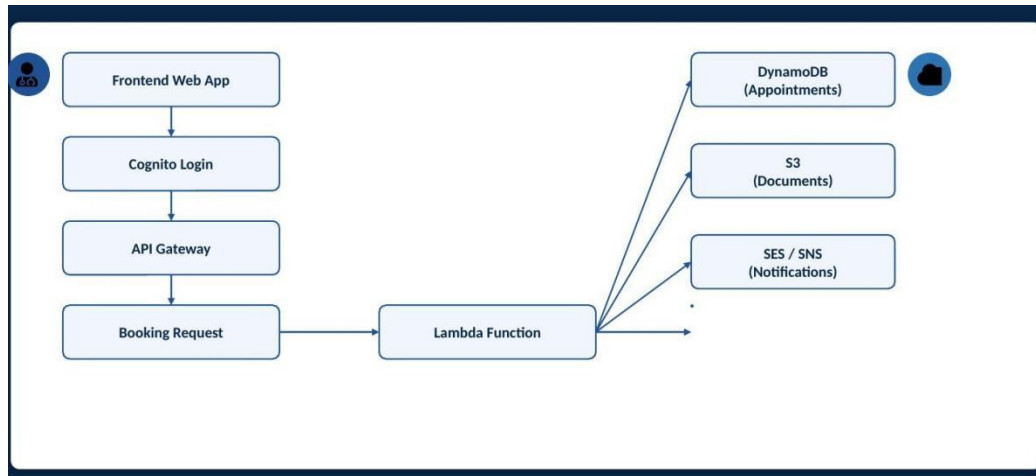


Figure 1. Overall system architecture of the Healthcare Appointment Booking System

4.1. Presentation Layer – Frontend Web App

The frontend web application is the entry point for patients and doctors. It provides an intuitive interface for account registration, doctor search, slot selection, and appointment management. Every request from the frontend is routed securely through the backend services described below.

- Supports patient and doctor account creation and login
- Displays real-time doctor availability by specialty
- Allows one-click booking and appointment history review

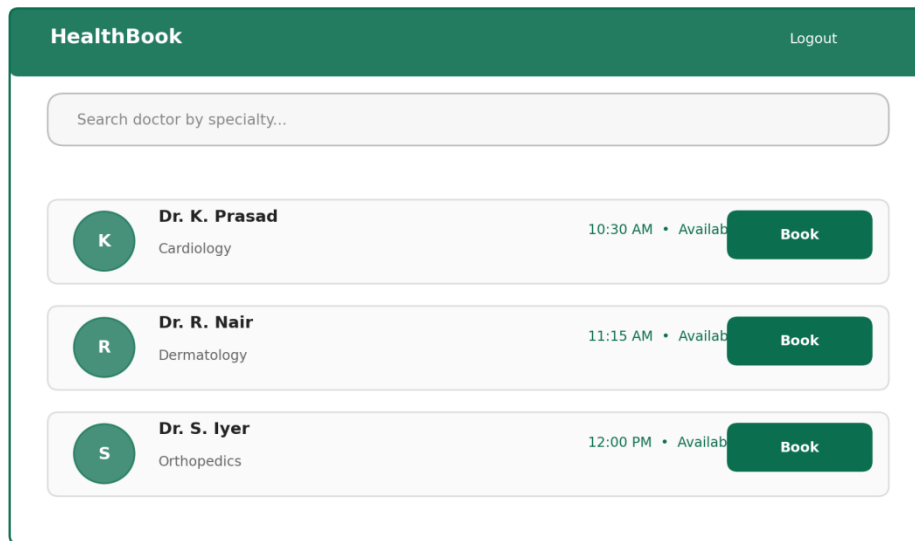


Figure 2. Frontend web application — doctor search and booking screen

4.2. Authentication Layer – Amazon Cognito

Amazon Cognito manages secure sign-up and sign-in for both patients and doctors. It verifies user identity before any booking request is processed, ensuring that only authenticated users can access or modify appointment data.

- Secure registration and login for patients and doctors
- Issues tokens used to authorize every subsequent API request
- Protects sensitive patient and medical data from unauthorized access



On every subsequent request, the token is attached and re-validated before Lambda is invoked

Figure 3. Amazon Cognito authentication flow

4.3. API Layer – Amazon API Gateway

Amazon API Gateway acts as the single entry point connecting the frontend web application to the backend business logic. Every booking request, search query, and profile update passes through API Gateway, which routes it to the appropriate Lambda function.

- Exposes secure REST endpoints for all patient and doctor actions
- Validates and routes incoming booking requests to AWS Lambda
- Works with Cognito to ensure only authenticated requests are processed

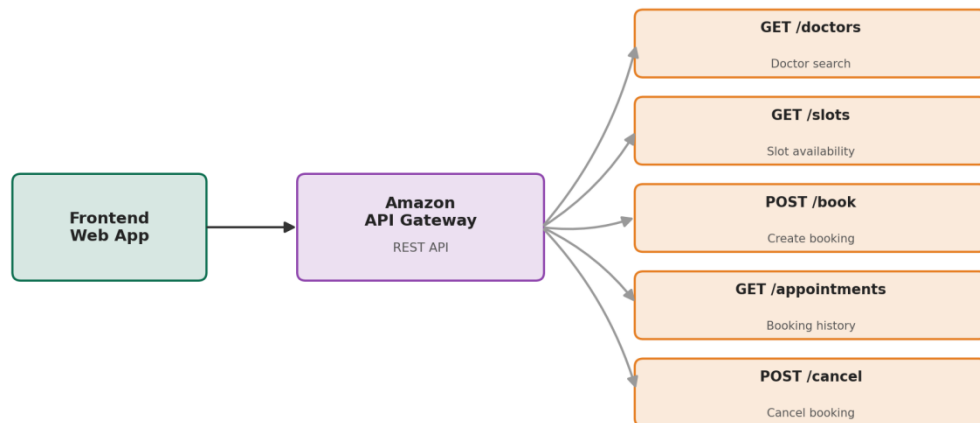


Figure 4. Amazon API Gateway endpoint routing

4.4. Processing Logic – AWS Lambda

AWS Lambda executes the core business logic of the platform. When a patient submits a booking request, a Lambda function performs several tasks:

- Validates the requested doctor, date, and time slot
- Checks DynamoDB to confirm the slot has not already been booked
- Writes the confirmed appointment record to DynamoDB
- Triggers a notification through Amazon SES/SNS to confirm the booking



AWS Lambda (Python 3.12) executes this sequence for every booking request

Figure 5. AWS Lambda booking-processing sequence

4.5. Data Storage – Amazon DynamoDB and Amazon S3

Amazon DynamoDB stores all patient, doctor, and appointment records, including AppointmentID, PatientID, DoctorID, Date, TimeSlot, and Status. Its fast, scalable reads and writes ensure that availability checks and booking confirmations happen in real time. Amazon S3 complements DynamoDB by storing supporting documents such as medical reports and profile photos.

- DynamoDB: fast, scalable NoSQL storage for appointments and user profiles
- S3: secure cloud storage for documents and profile photos
- Both services scale automatically with the number of users and bookings

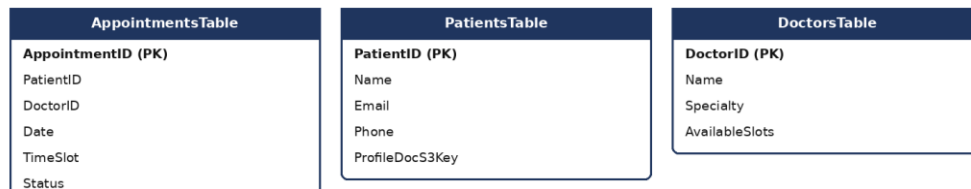


Figure 6. Simplified DynamoDB table schema for appointments, patients, and doctors

4.6. Notification Layer – Amazon SES / SNS

Once an appointment is confirmed, the Lambda function formats a confirmation message and publishes it through Amazon SES (for email) and Amazon SNS (for SMS). This ensures patients and doctors are notified instantly, without any manual follow-up from hospital staff.

- Sends instant email confirmations via Amazon SES
- Sends instant SMS confirmations via Amazon SNS
- Reliable, real-time, push-based delivery to patients and doctors

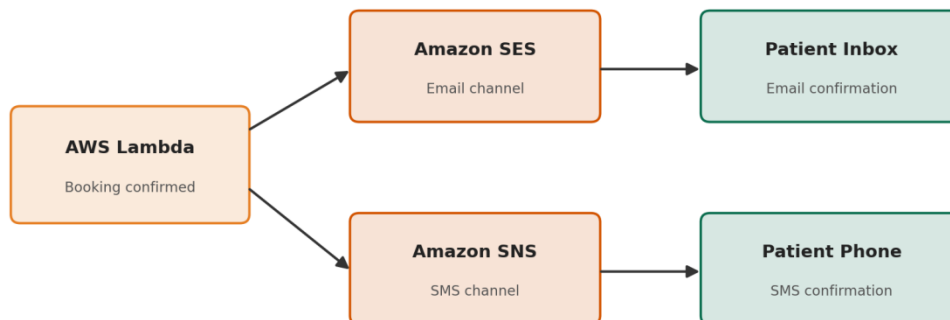


Figure 7. Amazon SES / SNS notification fan-out

4.7. Security and Monitoring – AWS IAM and Amazon CloudWatch

AWS IAM secures every interaction between services in the architecture, ensuring Lambda functions can only access the specific Cognito, DynamoDB, S3, and SES/SNS resources they require, following the principle of least privilege. Amazon CloudWatch continuously monitors Lambda execution, capturing logs and metrics that help the team detect and resolve issues quickly.

- IAM least-privilege roles govern every service-to-service interaction
- CloudWatch Logs capture execution details for debugging
- CloudWatch Metrics track invocation count, duration, and error rates

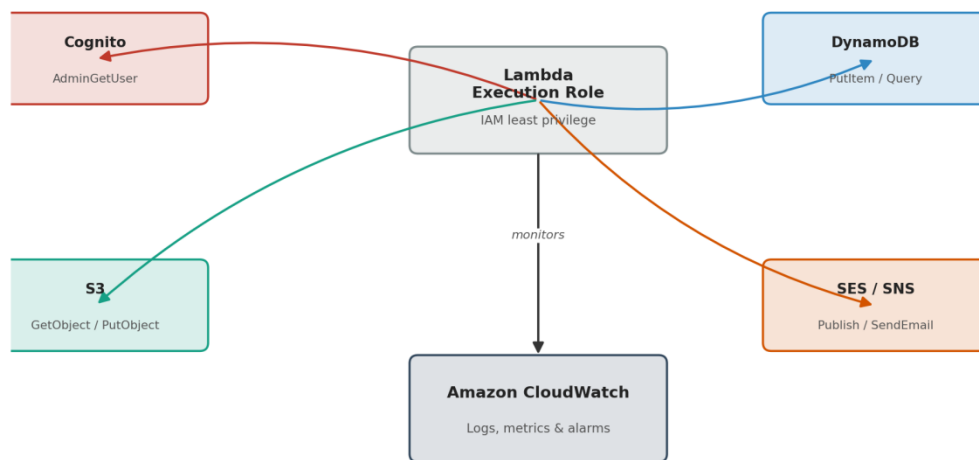


Figure 8. IAM execution role and CloudWatch monitoring

5. Implementation

The implementation of the Cloud-Based Healthcare Appointment Booking System translates the architectural design into a fully functional cloud-based application. This section outlines how each component was configured and developed to work together in a secure, automated booking workflow.

5.1. Patient and Doctor Registration & Login

Patients and doctors register through the frontend web app, with Amazon Cognito managing secure account creation and login. Cognito issues authentication tokens that are validated on every subsequent request, ensuring that only verified users can search for doctors or manage appointments.

5.2. Doctor Search & Slot Availability

Once logged in, patients can search for doctors by specialty. A Lambda function queries DynamoDB to retrieve the doctor's schedule and returns the currently available time slots, giving patients real-time visibility into open appointment times without needing to call the hospital.

5.3. Appointment Booking Logic

When a patient selects a slot, the request is routed through API Gateway to a booking Lambda function. Key steps include:

- Validating the requested doctor, date, and time slot.
- Checking DynamoDB to confirm the slot is still available.
- Writing a new appointment record with PatientID, DoctorID, Date, TimeSlot, and Status.
- Triggering a confirmation notification via SES/SNS.



Figure 9. End-to-end appointment booking workflow

5.4. Conflict Detection

To avoid double-booking, the Lambda function performs a conditional check against DynamoDB before confirming any appointment. If two patients attempt to book the same slot simultaneously, only the first valid request is accepted, and the second is prompted to choose another available time.

5.5. Notification Delivery

Once a booking is confirmed, the system automatically sends a confirmation message to the patient (and doctor, where applicable) via Amazon SES for email and Amazon SNS for SMS, ensuring both parties are informed instantly without manual intervention from hospital staff.

5.6. Data Storage & Security

All records are stored securely in Amazon DynamoDB, with supporting documents held in Amazon S3. IAM roles restrict each Lambda function to only the permissions it needs — for example, the booking function can write to DynamoDB and publish to SNS, but cannot access unrelated resources.

Key implementation details:

- Authentication: Amazon Cognito user pools for patients and doctors
- Backend runtime: Python 3.12 on AWS Lambda
- IAM roles scoped to each Lambda function's specific responsibilities
- Logging enabled via Amazon CloudWatch for all Lambda executions

The successful implementation of this system showcases how AWS managed services can be combined to create a secure, real-time, and low-maintenance healthcare appointment platform, with a modular design that supports future growth.

6. Results

The implementation of the Cloud-Based Healthcare Appointment Booking System yielded outcomes that align closely with the project's objectives. The system was evaluated across multiple scenarios to validate functionality, performance, scalability, and reliability of the booking workflow.

6.1. Successful Registration and Secure Login

Patients and doctors were able to register and log in reliably through Amazon Cognito, with authentication tokens correctly restricting access to booking and profile features to verified users only.

6.2. Real-Time Doctor Search

Doctor search queries returned accurate, up-to-date slot availability by reading directly from DynamoDB, giving patients an immediate and reliable view of open appointment times.

6.3. Conflict-Free Instant Booking

The conditional check built into the booking Lambda function successfully prevented double-booking in all tested scenarios, including simulated simultaneous booking attempts for the same doctor and time slot.

6.4. Automated Notifications

Confirmation emails and SMS messages were delivered to patients almost instantaneously after a successful booking, confirming that the Amazon SES/SNS notification pipeline is reliable and timely.

6.5. Scalable, Serverless Performance

The system handled varying volumes of booking requests without manual scaling or downtime, thanks to the underlying serverless AWS infrastructure. Testing scenarios included:

- Multiple patients booking appointments concurrently
- Repeated searches across different doctor specialties
- Monitoring Lambda invocation logs and response latency

All scenarios were handled successfully without performance degradation.

6.6. Secure and Controlled Access

Using IAM roles and Cognito-based authentication, access to patient data, appointment records, and notification services was tightly controlled, validating that the system follows best practices for healthcare data security.

6.7. User Experience

The system requires minimal effort from patients: registering once, searching for a doctor, and booking a slot in a single click. All subsequent steps — conflict checking, confirmation, and notification — are fully automated, confirming the platform's readiness for real-world deployment across hospitals and clinics.

The overall results demonstrate that the Cloud-Based Healthcare Appointment Booking System meets its intended goals of automation, security, scalability, and ease of use, with a stable architecture ready for real-time operational use.

My Appointments				
Appointment ID	Doctor	Department	Date & Time	Status
APT-20260718-0142	Dr. K. Prasad	Cardiology	18-Jul-2026, 10:30 AM	Confirmed
APT-20260722-0087	Dr. R. Nair	Dermatology	22-Jul-2026, 11:15 AM	Confirmed
APT-20260615-0033	Dr. S. Iyer	Orthopedics	15-Jun-2026, 12:00 PM	Completed

Patients can view, download, or cancel any upcoming appointment directly from this screen.

Figure 10. Sample output — My Appointments screen

7. Conclusion

The successful development of the Cloud-Based Healthcare Appointment Booking System demonstrates how a secure, automated, and cloud-native platform can eliminate long queues and manual scheduling errors in healthcare settings. By leveraging Amazon Cognito, API Gateway, Lambda, DynamoDB, S3, SES/SNS, IAM, and CloudWatch, the system achieves its core objectives: secure patient/doctor authentication, real-time doctor search, conflict-free instant booking, and automated notifications — all without managing any physical infrastructure.

The project highlights the benefits of serverless computing, including automatic scaling, cost-efficiency, and reduced operational complexity, while significantly improving the patient experience and hospital operational efficiency compared to traditional walk-in and phone-based booking.

Beyond its current functionality, the project lays a strong foundation for future enhancements, including:

- **Mobile App:** Native mobile apps for patients and doctors on iOS and Android.
- **Telemedicine:** Integrated video consultation for remote doctor visits.
- **AI Recommendations:** AI-based doctor recommendations based on symptoms and patient history.
- **Payment Gateway:** Secure online payment integration for consultation fees.
- **Multi-Hospital Support:** Extending the platform across multiple hospitals and branches.

In conclusion, this project serves as a practical, scalable blueprint for cloud-native healthcare applications, showcasing how AWS services can be strategically combined to build a secure, automated, and patient-friendly appointment booking solution.